Simulation-Based Bayesian Estimation of Dynamic Models

Cameron Fen*

October 10, 2021

This paper introduces a simulation-based deep learning approach to Bayesian inference to the economic literature allowing the estimation of posteriors even when the likelihood function is not tractable. This approach works even with data that has a latent time structure which is ubiquitous in macroeconomic structural modeling. This hasn't been successfully done econometrically, as far as I know, even in the simpler and easily extendable maximum likelihood setting. This approach allows for a general approach to Bayesian and MLE estimation of models solved via, for example, value function iteration and projection method as these methods don't and often don't, respectively, lead to tractable likelihood function. I demonstrate the validity of the approach by estimating two 4 parameter toy models and two research scale 10+ parameter models. Of the larger models, I estimate a 10 parameter HANK model solved via the Reiter method that generates 861 covariates per time step which shows that this approach can also handle very high dimensional data in terms of the simulated data. I also estimate an 11 parameter model solved via value function iteration, which I think is the first time a general value function iteration model has been estimated in a fully Bayesian fashion. Both models show that this approach can work for reasonably sized models used in the literature.

JEL Codes: C11, C68, C63, C45
Keywords: Neural Networks, Bayesian Inference, Dynamic Estimation, Simulation-Based Estimators

*Cameron Fen is PhD student at the University of Michigan, Ann Arbor, MI, 48104 (E-mail: camfen@umich.edu. Website: cameronfen.github.io.). The author thanks Eric Jang and Florian Gunsilius for helpful feedback and Alisdair McKay, Toni Whited and Emil Lakkis for code. All errors are my own.

I. Introduction

Much of the dynamic model literature in macroeconomics is focused on calibration or limited information estimation techniques. Bayesian estimation can generally only be used when the likelihood is defined, which is difficult for estimation techniques like value function iteration. Even if the likelihood can be worked out, Bayesian estimation is complicated to code and computationally expensive. However, the machine learning literature has developed a host of techniques for near efficient estimation of models without a likelihood function. GANs (Goodfellow et al., 2014), Variational Autoencoders (Kingma and Welling, 2013), and Normalizing Flows Rezende and Mohamed (2015) all are machine learning techniques that are robust to misspecification and also close to or are statistically efficient. Recently, these techniques have been started to be used in economics as methods that are both robust and efficient. In particular, Kaji, Manresa and Pouliot (2020) proposed to use GANs to get robust and near efficient estimation of structural models, using only simulations from the model and not a likelihood to perform estimation.

This papers main contribution is to introduce to the economics literature a set of three algorithms that can perform simulation based estimation of many economic models with this paper predominantly focus on dynamic models with latent time structure in a Bayesian setting, with easy extension to the maximum likelihood setting. Although the methodology is a bit different, this extends Kaji, Manresa and Pouliot (2020) approach to Bayesian inference to dynamic models with a latent time step component. To my knowledge, this is the first paper to be able to accomplish this and even accomplish dynamic model estimation with latent structure in the easier maximum likelihood setting, although there have been working paper attempts (Brownlees, Kristensen and Shin, 2011). The first algorithm, Sequential Neural Ratio Estimation (SNRE) Durkan, Murray and Papamakarios (2020), extends the GAN approach as introduced to economics by Kaji, Manresa and Pouliot (2020), to the Bayesian setting. A simulator is used to simulate from the joint distribution $X, \theta \sim P(X|\theta)P(\theta)$, then a GAN is used to evaluate the likelihood ratio, $P(X|\theta)$ from these samples. A second approach, Sequential Neural Likelihood Estimation (SNLE) (Papamakarios, Sterratt and Murray, 2019), uses a model relatively unknown in economics, the Normalizing Flow

Rezende and Mohamed (2015), as a powerful conditional density estimator, to estimate the posterior $P(\theta|X)$ on samples from the joint. One can think of SNLE as an extension of the Kristensen and Shin (2012) approach to likelihood estimation, except the normalizing flow replaces the kernel density estimator and extends there result to full Bayesian inference. The third approach, Sequential Neural Posterior Estimation (SNPE) (Greenberg, Nonnenmacher and Macke, 2019), replaces the MCMC sampling of θ in SNLE with a importance sampling approach which is faster and shares similarities with Herbst and Schorfheide (2014) and the general SMC^2 approach (Chopin, Jacob and Papaspiliopoulos, 2013).

The first advantage of this family of models is that the likelihood is not needed for full Bayesian estimation. Like Simulated Method of Moments (McFadden, 1989), one only needs to be able to sample from the model. This would allow techniques like value function iteration and more general approaches that are higher order (Fernández-Villaverde and Rubio-Ramírez, 2007) to be estimated in a Bayesian fashion. Second, computational cost is comparable if not faster than Markov Chain Monte Carlo (MCMC) with the ability to trivially parallelize the expensive simulation steps. The posterior of an RBC model can be derived in a couple of hours on a single computer, for the fastest method, SNPE. Furthermore, it can handle exotic and multimodal posteriors. Often maximum likelihood is more popular than a full Bayesian approach. Although there are likely faster gradient based extensions, a trivial extension of these models to maximum likelihood is to use the Maximum A Posterior (MAP) estimator with an additional improper prior. However there aren't any computational speed ups when using a MAP approach. Like MCMC, one needs to simulate up to 1 million models in large cases, but this can be trivially parallelized with no decline in computational efficiency, unlike MCMC. Lastly, the method conveniently separates the role of the macroeconomist from the econometrician. How the model is solved for simulation can be entirely opaque for econometricians constructing the estimation algorithm. Likewise, macroeconomists can focus on solving and simulating models without worrying about estimation. This is in contrast with likelihood and other gradient approaches which require the econometrician to have expertise in macroeconomic model being estimated. These and other advantages of simulation based inference has quickly made the technique one of the leading estimation methods in, for example, many fields

of physics (Brehmer, 2021), (Cranmer, Brehmer and Louppe, 2020).

The paper proceeds as follows: Section II. reviews the literature on simulation in economics and solving dynamic models; Section III. introduces the simulation based inference techniques that I will use; Section V.A. extends this description and discuss theory behind these techniques–additional proofs are in the appendix; Section VI. briefly discusses the economic models used to test the algorithms; Section VII. illustrates the results of the of the algorithm on the economic models; Section VII.; and Section VIII. concludes the paper.

II. Literature Review

This literature review will cover two topics: the literature in economics on simulation based likelihood and Bayesian inference techniques, and solving dynamic models. I will discuss machine learning background in the methods section following this one.

Most of the simulation based likelihood and inference approaches are inspired by an approach like Method of Simulated Moments (MSM) (McFadden, 1989), (Pakes and Pollard, 1989), (Duffie and Singleton, 1990). Due to the difficulty of verifying the global identification criterion there is a large interest in developing more efficient and robust estimators where this was not a problem. Techniques like maximum simulated likelihood, attempt to address this (Lerman and Manski, 1981), although these techniques have both have simulation bias (Hajivassiliou et al., 1997), (Haan and Uhlendorff, 2006) and cannot be estimated, at least easily, on data with a latent time structure. The premises is to basically simulate from the shock distributions and calculate the likelihood via this simulation and a Monte Carlo integral. There has also been work on using techniques from simulation based likelihood estimation to solve dynamic models, particularly in industrial organization (Keane and Wolpin, 1994). There is also literature on efficient simulation techniques using both indirect inference (Smith Jr, 1993), (Gourieroux, Monfort and Renault, 1993), and efficient method of moments (Gallant and Tauchen, 1996). While these techniques are efficient when the model is well specified, there has also been some work on robust and efficient method of moment estimators using a spectrum of moments so under some regularity conditions the data is guaranteed

to be in the support of distributions spanned by the moments[1] (Carrasco et al., 2007), (Altissimo and Mele, 2009). Additionally, like all the other approaches suggested, these methods have a difficult time dealing with models with latent time structure. Another potential approach is approximate Bayesian inference (Rubin, 1984)to estimate models solved via value function iteration techniques, to estimate these models. However in a well specified model, this approach is only approximately Bayesian, unlike the set of algorithms I will introduce to the economics literature, and is even more biased in misspecified models. One could even consider a particle filter a simulation based technique for deriving a likelihood function given an intractable integral (Fernández-Villaverde and Rubio-Ramírez, 2007), but this approach only works with dynamic models that yield complementary state space representations and requires a likelihood function at each point in time. Although the literature is more sparse, there are papers that also perform simulated Bayesian inference (Flury and Shephard, 2011), and this paper extends particle filtering for Bayesian Inference. Most relevant to this paper, Kaji, Manresa and Pouliot (2020) proposed a GAN neural network approach to get robust and near efficient estimation of structural models, using only simulations from the model and not a likelihood to perform estimation. This paper most closely builds off that, and provides models that can handle both Bayesian inference and latent time series structure.

There are a variety of ways to solve dynamic macroeconomic models: perturbation, projection, and value function iteration (Fernández-Villaverde, Rubio-Ramírez and Schorfheide, 2016), (Judd et al., 2017). There are other methods like Reiter's method (Reiter, 2009) and time iteration (Rendahl, 2017). Both these methods have advantages for solving state of the art heterogeneous agent model. Although drawbacks like requiring linearity for time iteration are important to consider. For a large set of models, almost anything that is solved via value function iteration (Hennessy and Whited, 2007), heterogeneous agent models with intractable likelihoods (Kukackaa and Barunika, n.d.), many projection models, and models with multiple error terms, the likelihood is intractable and so in order to estimate these models in a Bayesian manner, one needs to simulate from the model. As such, most of the time, these models are not estimate in a Bayesian or even full infor-

1. There is also work in machine learning attempting to perform robust and efficient moment estimation by using a spectrum of moments, most famously the Generative Moment Matching Network (Li, Swersky and Zemel, 2015)

mation manner and practitioners resort to method of simulated moments. This paper shows how to estimate small and medium sized models solved via value function iteration in a Bayesian fashion using simulation. This approach also works for models solved via projection with intractable likelihoods. This paper also shows how to estimate small and medium sized heterogeneous agent models using simulation. Although the models are solved via time iteration which allows MCMC estimation of the posterior for comparison, the algorithm can handle data output by the model which has 821 covariates (different quantiles of the state distributions), this suggests that the approach can handle heterogeneous agent models that are also high dimensional, but may not have tractable likelihoods.

III. Background on Simulation Neural Ratio Estimators

III.A. General Approach of Simulation Based Inference

When discussing the approach of Simulation-based inference, it is useful to lay down a few definitions. The true data comes from the underlying data generating process. For macroeconomics, it would be economic data like output, consumption, etc. The simulator is the model being estimated, ie for this paper generally a dynamic macro model. The only demands SBI puts on simulators is to be able to simulate data that has a 1-1 correspondence to the true data, outputwise. Thus, all that SBI demands from the macroeconomist is to provide a model that is solved and can be simulated from for any parameter drawn from the proposal distribution.

The fundamental insight behind SBI is that for most models you can draw as many samples from the joint distribution of $X, \theta \sim P(X, \theta)$ as one wants, given an appropriate simulator. Once you have these joint samples, you can estimate the posterior distribution by fitting a conditional density estimator $P(\theta|X)$ on these samples, which when conditioned on the real data is the posterior. The differences between the three SBI approaches roughly centers around what conditional density estimator to use. SNRE uses a GAN, SNLE and SNPE use a Normalizing Flow. While one can just sample a lot of points from the joint, an iterative approach where one fits the density estimator on sampled data then re-samples $\theta \sim P(\theta|X)$ on the X that is the observed data time series and repeat

rounds of estimation allows for much faster convergence as the joint space around the observed X is the space where sampling is focused. The sbi library handles the entire estimation process and other than some relatively boilerplate code the main problem left to the macroeconomist is to build the simulator. Solving a model so one can simulate from it is a tool most macroeconomist are comfortable with. The use of the normalizing flow or GAN can be handled in the background without macroeconomist input.

While all three models can be used for simulation-based Bayesian inference, this results section will focus on using SNPE, as this technique is both the fastest and most effective way to perform Bayesian Estimation of models especially with latent dynamics. I also will also have some results, corroborating SNPE, using SNRE. SNLE can handle many simulated Bayesian inference settings, however it is difficult to get SNLE to work in the specific scenario of models with latent variables across time. The proofs in the appendix regarding universal approximation and bijectiveness of normalizing flows will focus on SNPE. Similar proofs of universal approximation can apply for GANs in SNRE and is easily extended from the celebrated original universal approximator result for feed forward neural networks (Cybenko, 1989).

III.B. Generative Adversarial Networks

The GAN is an algorithm that uses two competing models to moving in tandem to generate data that matches data generated from the real world (Goodfellow et al., 2014). Below is a diagram describing the structure of a basic GAN:

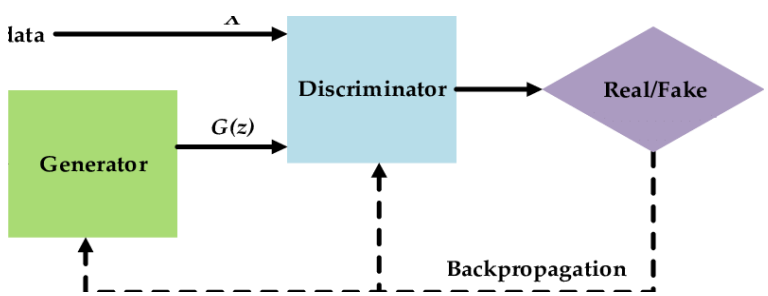

Figure I: Diagram of a GAN

The generator is a neural network (or any differentiable model like a dynamic macro model) which attempts to produce data (unconditionally or conditionally via some inputs X) that resembles data in the real world. The discriminator is another neural network (or any trainable model that can produce a logistic output), which outputs the probability a given input data comes from the underlying data generating process and not from the generator. The two networks play a game where the objective of the generator is to convince the discriminator to assign high likelihood to its output and the objective of the discriminator is to assign the correct probabilistic label, a number between 0 or 1, to input that is either real data or generated data. One way to think about a GAN is a changing simulated method of moments system where the moment conditions automatically evolve over many rounds to pick the most mismatched moments between the true data and the model. When thought in this way, it's intuitive to recognize that the GAN is nearly a full information and efficient estimation procedure.

More mathematically, the objective of the discriminator is to minimize a logit loss of input that comes either from real data x, or the generator $G(z)$ where the z are shocks that ensure the support of G includes all x. Mathematically the discriminator loss is:

$$Loss_D = -E_{x \sim P_{data}(x)}[D(x)] - E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{1}$$

This is just a logit loss applied to data either coming from the data generating process or the generator. Likewise the generator's loss is to fool the discriminator:

$$Loss_G = E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{2}$$

These two loss functions can be compactly represented as:

$$G, D = \min_G \max_D E_{x \sim P_{data}(x)}[D(x)] + E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{3}$$

Equation (3) makes it clear the GAN objective is a minimax game with a solution that is a Nash equlibrium. However what is important for simulation based estimation is to recover likelihoods/probabilities. The next proposition following Goodfellow et al. (2014) will show how to do that:

Proposition 1.: For a fixed generator, the optimal discriminator will produce a logistic output:

$$D*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_{generator}(x)} \tag{4}$$

Proof Using equation (1), for the loss of the generator:

$$Loss_D = -\int_x log(D(x))p_{data}(x)dx - \int_z log(1 - D(G(z)))p_{generator}(z)dx \tag{5}$$

$$= -\int_x (log(D(x))p_{data}(x) - log(1 - D(x))p_{generator}(x))dx \tag{6}$$

(7)

Any discriminator that minimizes the integral: $-\int_x (log(D(x))p_{data}(x) - log(1-D(x))p_{generator}(x))dx$, minimizes the integral pointwise over all x. Thus the optimal discriminator will set the first order condition of the intergrand to zero:

$$0 = \frac{p_{data}(x)}{D*(x)} + \frac{p_{generator}(x)}{1 - D*(x)} \tag{8}$$

Rearranging equation (8) gets the result of the proposition.

Then one can use this proposition along with the discriminator output to estimate a likelihood ratio.

III.C. Sequential Neural Ratio Estimation

For one verison of SNRE, The authors Durkan, Murray and Papamakarios (2020) use a GAN, not to match a underlying data like in proposition 1, but to contrast between two different distributions to estimate a density. In particular, the authors sample from the distribution $x, \theta \sim p(x)p(\theta)$ where is sampled uniformly from the data and θ is the prior. Then they contrast the independent distribution, with the correlation between X and θ broken with the joint simulated data $x, \theta \sim p(x, \theta) = p(x|\theta)p(\theta)$, where $x \sim p(x|\theta)$ is generated from the simulator. Furthermore, the authors recognize the ratio $r(x, \theta) = \dfrac{p(x, \theta)}{p(x)p(\theta)}$, easily derived from the optimal discriminator, is equivalent to $\dfrac{p(\theta|x)}{p(\theta)}$. Thus, the density of the posterior $p(\theta|x) = r(x, \theta)p(\theta)$, where $p(\theta)$ is the prior.

However when estimation occurs over multiple rounds, one has to be able to sample θ from the proposed posterior to generate additional samples from the joint to further refine the GAN and refine the proposed posterior. This is accomplished using Metropolis-Hastings MCMC sampling,

which allows sampling from the posterior knowing only unnormalized likelihood.

Algorithm 1: SNRE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, discriminator $d_\phi(x,\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with MCMC;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(x,\theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Sample K contrasting samples for each i sample $\{x'^{(k)}\}_k^{B*K}, \{\theta'^{(k)}\}_k^{B*K} \sim D\{x\}, D\{\theta\}$, breaking the joint distribution of D into independent marginals;

    Train the GAN using the GAN loss function from the two distributions x, θ and x', θ' and a softmax/multinomial logit objective $\sum_i^B log(\frac{exp(d_\phi(x^{(i)}, \theta^{(i)})}{\sum_k^K exp(d_\phi(x^{(k)}, \theta^{(k)})})$;

  end

  Update posterior $p^{(i)} \propto exp(d_\phi(x,\theta))p(\theta)$;

end

Now as highlighted in Kaji, Manresa and Pouliot (2020), GAN inference doesn't quite return the maximum likelihood solution. In fact, while the maximum likelihood chooses parameters that minimizes the KL divergence between the data distribution with the distribution of the models, GAN inference minimizes the Jensen-Shannon divergence[2] between the data distribution and the model. Empirically the difference is not large and if the model is well specified the maximum likelihood optimal is the same. However, this does carry over when using GAN inference for Bayesian estimation. While empirically this difference is often small so that it's glossed over in many machine learning papers (Tran, Ranganath and Blei, 2017), (Durkan, Murray and Papamakarios, 2020), (Hermans, Begy and Louppe, 2020), there are ways to truly perform likelihood estimation

2. i.e. 1/2 times the forward KL divergence and the reverse KL divergence

with a GAN like structure (Cranmer, Pavez and Louppe, 2015). This will be discussed further in the more theoretical treatment of SNRE below.

IV. Background on Simulation Neural Likelihood Estimation and Simulation Neural Posterior Estimation

IV.A. Normalizing Flows

sectionNormalizing Flows A normalizing flow is a neural network that stacks together many invertable changes-in-measure. Given a density, $p(x^i)$, with i and j indexing the vector x; a bijector, $f(x^i)$, has density, $q(x^j)$ such that the probability $q(x^j)$ is related to the probability $p(x^i)$ via:

$$q(x^j) = p(x^i)|\frac{df^{-1}}{dx^i}| = p(x^i)|\frac{df(x^i)}{dx^i}|^{-1} \tag{9}$$

This is the traditional change of variables formula in calculus or in a first year stats/econometrics course (**?**). More formally, this is the Radon-Nikodym derivative for a change in measure, although it requires bidirectional absolute continuity because I want to apply the change of variable transformation in both directions (ie f and f^{-1}). Because the transform is a bijection, x and y have to be the same dimensions[3]. Using a convenient base distribution x^i, like a Gaussian, one can then "stack" many of these bijectors to form a flexible density estimator that transforms x^i via intermediate variables x^j, x^k... to the target variable x^n.

The main difficulty left is to define an f such that it can be inverted easily and for higher-dimensional applications, has a Jacobian, $\frac{df(x)}{dx}$, that can be computed efficiently. One set of invertable transformations are affine operations. Given $x^i \in R^N$ and a going from 1..N indexing each element in the vector x^i:

3. See, for example, Cunningham et al. (2020) for relaxations

$$\forall a, x_a^j = \beta_a^0 + \beta_a^1 * x_a^i \tag{10}$$

This operation takes each element in the vector, x_a^i, and scales and shifts the element to return the corresponding element of the target space vector, x_a^j. This is trivially invertable by shifting by the negative and scaling by the inverse. While one can still add nonlinearities between bijective couplings, this still lacks expressivity. The standard way to extend this model to allow for the shifts and scales to be dependent on at least some of the inputs is to transform some elements of the input vector, for example $x_{a:l}^i$, via only the identity (ie $x_{a:l}^i = x_{a:l}^j$), then condition the shifts and scales for the rest of the elements $x_{l:N}^i$ on inputs, $x_{a:l}^i$, that weren't transformed:

$$\forall \iota \in l : N, x_\iota^j = \beta_\iota^0(x_{a:l}^i) + \beta_\iota^1(x_{a:l}^i) * x_\iota^i \tag{11}$$

$\beta_0(.)$ and $\beta_1(.)$ are typically differentiable and flexible functions–typically feed-forward neural networks. Since each affine transformation is a function of variables that have an index smaller than the index in question, the Jacobian is lower triangular and the determinant is just the product on the diagonal. Furthermore, as a permutation of indices is a bijection, one can condition on arbitrary subset of parameters as one stacks these bijections on top of one another. If the the functions for $\beta_0(.)$ and $\beta_1(.)$ are universal approximators (ie neural networks), one can show that a stack of bijectors (along with permutations), is also a universal approximator and thus can approximate any bijection arbitrarily well (Huang et al., 2018). This will be discussed and proved in the theoretical section on normalizing flows.

One can also condition $\beta_0(.)$ and $\beta_1(.)$ on arbitrary conditioning variables to form a flexible conditional density estimator. Since these models can model any conditional distribution, and can calculate the likelihood of any sample point in the target space, one can fit this model on samples from arbitrary continuous densities and have the guarantees that come with a well specified maximum likelihood problem. Thus, normalizing flows are density estimators that are robust to

misspecification, asymptotically efficient due to maximum likelihood, and internally consistent as a single model transforms, for example, normal noise to arbitrary distributions on R^N. While sample convergence rates are not known for normalizing flows, each bijection layer is a combination of a (ReLU) feedforward network combined with an affine transformation of the output of the feedforward with the input and so the result likely be similar to the nearly root-n convergence rates for feed-forward networks discussed in Farrell, Liang and Misra (2018) as well as Barron (1993).

IV.B. Theoretical Background on Normalizing Flows

Since the SNPE will be the workhorse model of this paper, I will prove many more properties of normalizing flows. Additionally since a GAN has essentially the same architecture as most neural networks, the usual theorems like Cybenko (1989) hold without much modification of the proof. The normalizing flow has an entirely different architecture and even though I show that universal approximation does hold, the proof requires more work than the GAN case.

I first relay a lemma:

Lemma c: A neural autoregessive bijectors defined as: $y_i = \sum_k w_k(x_1, ...x_{i-1}) * \sigma(\alpha_k(x_1, ...x_{i-1}) * x_i + \beta_k(x_1, ...x_{i-1}))$ with α, β, and w being outputs from neural networks with all positive weights and a monotonic non-linearity, σ, is monotonically increasing with each bijector.

The proof is in the appendix.

Knowing this lemma one can prove this theorem regarding the bijectiveness of a neural autoregressive bijector:

Theorem 1: A bijector learns a bijection between the input and output space

The proof is again in the appendix.

Combining theorem 1 along with lemma a and lemma b in the appendix, I prove a theorem allowing one to calculate the pdf of the output of the normalizing flow:

Theorem 2: The chain of neural autoregressive bijectors along with a bijector that permutes the indices of inputs, learns a valid change in measure of a input base distribution to an output target distribution and the pdf of the target distribution can be derived using the standard change of variable formula: $q(y) = q(x)|det(df/dx)|^{-1}$.

Knowing this, one can perform the change of variables highlighed in the earlier section to fit a density estimator to samples via maximum likelihood. Moving on, one essential property of any density estimator is that it should be sufficiently flexible hopefully to be able to model any distribution. For example due to it's non-parametric nature and its flexibility to approximate any distribution given enough data, makes the kernel density estimator (KDE) the workhorse model for many density estimation problem. However because of difficulties estimating conditional densities, often one has to resort to quantile regression which lacks the non-parameteric flexibility of KDEs. It is easy to condition normalizing flows on external variables–one can just add the conditioning inputs into each neural network governing every α and β. However, I still need to prove that these models are flexible enough to model any distribution, which is the below theorem. First a few lemmas:

Lemma 1: Step functions universally approximate monotonic functions.

Lemma 2: I will move on to proving the same thing with sigmoid functions of the form:

$$S'(x) = \sum_k w_k * \sigma((x - b_k)/\tau_k)$$

Also can universally approximate arbitrary monotonic functions.

Lemma 3: Let the base and target space be multivariate and α, β, and w now be the outputs of neural networks conditioned on previous data. Bijectors parameterized like this:

$$y'_i = \sum_k w_k(x_1, ...x_{i-1}) * \sigma(\alpha_k(x_1, ...x_{i-1}) * x_i + \beta_k(x_1, ...x_{i-1})) \tag{12}$$

universally approximates monotonic functions.

Lemma 4: The neural network parametrized by

$$G(X)_t = \sigma^{-1}(\sum_k w_k(x_1, ...x_{i-1}) * \sigma(\alpha_k(x_1, ...x_{i-1}) * x_i + \beta_k(x_1, ...x_{i-1})))$$

can transform a base distribution of an m dimensional uniform random variables to any desired m dimensional random variable conditioned on inputs with an index lower than the current index

with arbitrary precision.

lemma 5

Given an arbitrary base distribution, a bijector can map this distribution to an arbitrary uniform distribution.

Combining all these lemmas, I construct a proof following Huang et al. (2018) that neural autoregressive normalizing flows are universal approximators of distributions:

Theorem 6

A stack of at least 2 bijectors in the form,

$$H(X)_t = \sigma^{-1}(\sum_k w_k(x_1, ...x_{i-1}) * \sigma(\alpha_k(x_1, ...x_{i-1}) * x_i + \beta_k(x_1, ...x_{i-1})))$$

with a bijector that sufficiently permutes indices values between each neural autoregressive bijector can map arbitrary distributions to arbitrary distributions.

This result finally proves that neural autoregressive normalizing flows like kernel density estimators can fit any distribution given arbitrary amounts of data and arbitrary size of these models.

IV.C. SNLE and SNPE

The SNLE and SNPE models differ from SNRE due to replacing the likelihood estimator with a normalizing flow conditional density estimator. Like in SNRE, one samples parameters from a proposed posterior distribution, then samples from the model conditional on the sampled parameters. This gets a set of data from the joint distribution $x, \theta \sim P(x, \theta)$ and fits a conditional normalizing flow on the joint distribution to get the likelihood $P(X|\theta)$. After estimating a the likelihood, one can recover a proposal posterior by multiplying by the prior: $p^{(i)} \propto P(X|\theta)P(\theta)$. Unlike SNRE, these two methods use an unbiased estimator for the likelihood (albeit in asymptotic running time),

when calculating the posterior (Papamakarios, 2019).

Algorithm 2: SNLE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, flow $f_\phi(x|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with MCMC;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(x, \theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $f_\phi(x|\theta)$ on $\{x^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)} \propto f_\phi(x|\theta)p(\theta)$;

end

With SNPE, one samples from the proposed posterior $p^{(i)}$ via MCMC. With SNPE, one samples from the proposed posterior estimates the posterior $p(\theta|x)$ not the likelihood like in SNRE. One then importance samples away the error of sampling theta from proposed posterior from the prior. More mathematically, given a proposed prior of $p'(\theta)$, true prior of $p(\theta)$, the posterior, when trained on this data, will have to be importance sample adjusted by the factor of $\frac{p(\theta)}{p'(\theta)}$, to account for sampling from a distribution that isn't the prior. The authors perform the estimation in one step by recognizing the adjusted distribution $p'(\theta|x) = p(\theta|x)\frac{p'(\theta)}{p(\theta)}$, where $p'(\theta|x)$ is the posterior obtained by sampling from a proposal distribution different then the prior. Then if a normalizing flow $f_\phi(\theta|x)$ is estimated in place of $p(\theta|x)$ in the above equation, a fully flexible $f_\phi(\theta|x)$ will return

a unbiased estimator of the posterior.

Algorithm 3: SNPE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, flow $f_\phi(x|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(x, \theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $f_\phi(\theta|x)\dfrac{p^{(i-1)}}{p(\theta)}$ on $\{x^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)} \propto f_\phi(\theta|x)$;

end

The SNPE algorithm is the fastest of the three algorithms because it does not use MCMC sampling. Tests that I and others have done (Greenberg, Nonnenmacher and Macke, 2019) suggests no sacrifice in accuracy either. As such, for most of the results, I will focus on the SNPE algorithm, although I perform some tests using the SNRE algorithm due to significance of GANs in the economics estimation literature especially due to their connection with method of moment techniques.

V. Theoretical Properties of Sequential Neural Estimators

This section will discuss the Bayesian properties of SNRE, SNLE, and SNPE. Ratio estimation will be discussed first as this is the most widely understood model for economists. All three models produce posteriors that are fairly accurate in simulations despite not performing exact Bayesian

inference in the misspecified case, the least "correct" is SNRE, but the main disadvantage of SNRE is not the accuracy but the higher computational cost.

V.A. Theoretical Properties of SNRE

Proposition 2 SNRE converges to the Bayesian Posterior *Assumptions:* 1. The parameters of the discriminator, $f_\phi(\theta; x)$ that optimally discriminates true $p(\theta|x)/p(\theta)$ from any other distribution is unique and in the interior of the space of θ 2. The simulations produced are smooth functions of the observables and are derived from parameters in the support of the prior

Proof: Since the discriminator in a GAN is a universal approximator, it can learn arbitrarily well the odds ratio of $\frac{p(X,\theta)}{p(X)p(\theta)}$, given enough simulated data. Since this approaches true likelihood, when multiplied by the true prior or an importance sample adjusted proposal distribution (see above or SNPE proof below for intuition and proof why this is true) this gives a pdf of the true joint distribution. Since MCMC also asymptotically draws samples from the true distrubtion, given unnormalized probability densities, MCMC will sample from the true posterior in the limit.

V.B. Theoretical Properties of SNLE and SNPE

Before proving theoretical properties of SNLE and SNPE, I first prove a lemma that maximizing the likelihood is equivalent of minimizing the KL divergence between the model and the data distribution. **Lemma 1**: Minimizing the KL divergence between the data and the model is equivalent to maximizing the likelihood.

(13)

$$\min_\theta KL(P(x)||Q_\theta(x)) = \min_\theta E_{P(x)}[log(P(x)) - log(Q_\theta(x)] \quad = \min_\theta -E_{P(x)}[log(Q_\theta(x)] = \max_\theta E_{P(x)}[log(Q_\theta(x)]$$

The second line is equal to the first because $P(x)$ is not a function of θ. The other qualities are the result of algebra or definitions.

Proposition 4 SNLE converges to the Bayesian Posterior *Assumptions:* 1. Absolutely Continuous Assumption: The support of the likelihood of the model is contained in the support of

the posterior. 2. Continuous Assumption: The simulations produced are smooth functions of the observables in the support of the model likelihood and prior

Proof:

Since the flow is performing MLE, it will pick parameters that that minimize the KL divergence between the flow and the correct likelihood distribution $p(x|\theta)$. Since the flow is a universal approximator, given enough data and a large enough flow, the true likelihood will be arbitrarily close, in a KL divergence sense, to the best parameterization of the flow. Thus, as the number of simulations goes to infinity and the parameters of the models go to infinity, the flow will converge to the true likelihood. Since this is the true likelihood, when multiplied by the true prior or an importance sample adjusted proposal distribution (see above or SNPE proof below for intuition and proof why this is true) this gives a pdf of the true joint distribution. Since MCMC also asymptotically draws samples from the true posterior, using the joint distribution as the unnormalized density and conditioning on X when MCMC sampling will draw samples from the true posterior.

Proposition 5 SNPE converges to the Bayesian Posterior *Assumptions:* 1. Absolutely Continuous Assumption: The support of the likelihood of the model is contained in the support of the posterior. 2. Continuous Assumption: The simulations produced are smooth functions of the observables in the support of the model likelihood and prior

Proof: Since multi-round SPNE samples from a proposal distribution, $p'(\theta|x)$, then fitting the following function on joint simulated data will result in importance sampling corrected posterior: $f_\phi(\theta|x)\frac{p'(\theta|x)}{p(\theta)}$, where f_ϕ is the flow. By definition of importance sampling, this is the correct posterior after adjusting the prior for the proposal distribution. Since the flow is performing MLE, it will pick parameters that that minimize the KL divergence between the flow and the correct posterior. Since the flow is a universal approximator, given enough data and a large enough flow, the true posterior will be arbitrarily close in a KL divergence sense, to the best parameterization of the flow. Thus as the number of simulations goes to infinity and the parameters of the models go to infinity, the flow will converge to the true posterior.

VI. Economic Models

The models used to test this algorithm are disparate. I estimate a relatively standard RBC model via time iteration, a model of capital formation solved via value function iteration dynamic programming, a HANK model solved via Rieter's method and time iteration and lastly a 11 parameter bequests model estimated via value function iteration. I will now discuss each of these models sequentially.

VI.A. The RBC Model

This RBC model comes from Alisdair McKay's note on Heterogeneous agent models: `https://alisdairmckay.com/Notes/HetAgents/index.html`.

The utility function is a standard CRRA utility:

$$u(C_t) = E\sum_t \beta^t \frac{C_t^{1-\gamma}}{1-\gamma} \tag{14}$$

The production function is standard Cobb-Douglas with productivity:

$$Y_t = Z_t K_t^{\alpha} \bar{L}^{1-\alpha} \tag{15}$$

Where $\bar{L}$ is fixed labor and Z_t is productivity and K_t is capital which evolves according to a standard law of motion:

$$K_t = (1-\delta)K_{t-1} + Y_t - C_t \tag{16}$$

Productivity evolves according to a AR(1) process in logs:

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{17}$$

VI.B. The Model of Capital Formation

This model comes from code written by Emil Lakkis in a class I took with Toni Whited. This model is a straightforward infinite horizon partial equilibrium model with risk neutral managers choosing case flow to maximize the present value of a stream of firm cash flows.

Utility/cash flows are give by this formula:

$$U(K_t, I_t, Z_t) = \sum_t \beta^t * (\pi(K_t, Z_t) - I_t) \tag{18}$$

Where K_t, Z_t, and I_t are the same as in the RBC model–capital, productivity and investment respectively. π is the profit function.

The capital law of motion is like the RBC model:

$$K_t = (1 - \delta)K_{t-1} + I_t \tag{19}$$

Productivity evolves according to a AR(1) process in logs:

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{20}$$

(is this right?) Profit is defined as:

$$\pi(K_t, Z_t) = Z_t K_t^{\alpha} \tag{21}$$

VI.C. The HANK Model

This model which has heterogeneous states, also comes from Alisdair McKay's tutorial. Preferences are still CRRA like the RBC model. The firm side is now new Keynesian with itermediate goods produced via:

$$y_{j,t} = Z_t N_{j,t} \tag{22}$$

Here Z_t is aggregate productivity and N_t is labor employed by firm j producing variety $y_{j,t}$. The final good is produced via the Dixit-Stiglitz aggregator Dixit and Stiglitz (1977):

$$Y_t = (\int_0^1 y_{j,t}^{\frac{\epsilon-1}{\epsilon}})^{\frac{\epsilon}{\epsilon-1}} \tag{23}$$

Here ϵ is the elasticity of substitution between intermediates.

Output is defined as,

$Y_t = A_t \int_0^1 n_{j,t} dj = A_t N_t$

Where $n_{j,t}$ is the labor for the individual intermediate goods. Additionally from the accounting identity, output is also, $Y_t = C_t + \psi M_t H_t$

Where the second term is hiring costs.

Government and bonds are defined by the following equations:

Labor market is a standard McCall Search model:

The household maximizes the CRRA utilty like the RBC model. The households budget constraint in real money is:

$$a_t + C_t = R_t * a_{t-1} + earnings \tag{24}$$

Here a_t is savings and R_t is the interest rate as determined by the bond market clearing condition. Earnings are given by:

$$\text{earnings} = \begin{cases} (1-\tau_t)(w_t + d_t) & \text{if employed} \\ b & \text{if unemployed,} \end{cases} \tag{25}$$

Here w_t is wages, d_t is dividend income and b is unemployment benefits.

The stochastic matrix governing employment, unemployment transitions is given by:

$$\begin{pmatrix} 1 - M_t & \delta \\ M_t & 1 - \delta \end{pmatrix} \tag{26}$$

Again M_t is governed by the labor market equations above.

The firm solves the Dixit-Stiglitz cost problem:

$$y_{j,t} = \left(\frac{p_{j,t}}{P_t}\right)^{-\varepsilon} Y_t \tag{27}$$

with price index P_t given by

$$P_t^{1-\varepsilon} = \int_0^1 p_{j,t}^{1-\varepsilon} dj. \tag{28}$$

The Calvo pricing adjustment parameter θ implies that the intermediate firm chooses price $p*$ to maximize:

$$\mathbb{E}_t \sum_{s=t}^{\infty} \theta^{s-t} R_{t,s}^{-1} \left[\frac{p_t^*}{P_s} y_{j,s} - w_s n_{j,s} - \psi M_s h_{j,s}\right] \tag{29}$$

Here $n_{j,t}$ denotes employment at firm j in time t. $h_{j,t}$ denotes hiring and $R_{t,s}^{-1}$ denotes the real interest rate discounted s periods back to time t.

VI.D. The Value Function Iteraton Bequests Model

The final model comes from the github page: `https://github.com/pkofod/vfi/blob/master/Fast%20VFI.ipynb`. The author, Jeppe Druedahl, wrote this code to solve models using value function iteration. I will use the example model in this github code. The utility function is:

$$u(b_t, c_t) = \frac{[\phi(b_t + \underline{b})^{1-\gamma} + (1-\phi)c_t^{1-\gamma}]^{\frac{1-\rho}{1-\gamma}}}{1-\rho} \tag{30}$$

Here b_t is bequest or non-liquid savings. c_t is consummation. b, ϕ, γ, ρ are all parameters. The

bequest utility function (end of life utility) is:

$$\nu(a_t, b_t) = \varphi \frac{(a_t + b_t + \underline{q})^{1-\vartheta}}{1-\vartheta} \tag{31}$$

Here a_t is liquid savings and q, θ, φ are all parameters. The value function is give by:

$$v_t(m_t, n_t, l_t) = \max\{v_t^{keep}(m_t, n_t, l_t), v_t^{adj}(m_t, n_t, l_t)\} \tag{32}$$

$$\text{s.t.} \tag{33}$$

$$x_t = m_t + (1-\tau)n_t \tag{34}$$

Here m_t is beginning of period liquid savings (analogous to end of period a_t) and n_t is beginning of period illiquid savings (analogous to end of period b_t). x_t is the pooled value of total liquid and illiquid savings. l_t is labor income. The post-decision value function is:

$$w_t(a_t, b_t, l_t) = \nu(a_t, b_t), t = T \tag{35}$$

$$w_t(a_t, b_t, l_t) = \mathbb{E}_t \left[\max\{v_t^{keep}(m_{t+1}, n_{t+1}, l_{t+1}), v_t^{adj}(x_{t+1}, l_{t+1})\}\right], t < T \tag{36}$$

$$\text{s.t.} \tag{37}$$

$$l_{t+1} \sim F(l_t) \tag{38}$$

$$m_{t+1} = (1+r_a)a_t + \omega l_{t+1} \tag{39}$$

$$n_{t+1} = (1-\delta)b_t \tag{40}$$

$$x_{t+1} = m_{t+1} + (1-\tau)n_{t+1} \tag{41}$$

Here labor income comes from a distribution $F(l_t)$. Likewise $r_a, \omega, \delta, \tau$ are all parameters. The

keep value function is:

$$v_t^{keep}(m_t, n_t, l_t) = \max_{c_t \in [0, m_t]} u(n_t, c_t) + \beta w_t(a_t, b_t, l_t) \tag{42}$$

$$\text{s.t.} \tag{43}$$

$$a_t = m_t - c_t \tag{44}$$

$$b_t = n_t \tag{45}$$

The keep value function is the value function at a point t if the agent chooses to consume and not withdraw or deposit funds into the illiquid asset, b_t. The adjust value function is:

$$v_t^{adj}(x_t, l_t) = \max_{b_t \in [0, x_t]} v_t^{keep}(m_t, n_t, l_t) \tag{46}$$

$$\text{s.t.} \tag{47}$$

$$m_t = x_t - b_t \tag{48}$$

$$n_t = b_t \tag{49}$$

The adjust value function is the value of adjusting the illiquid asset. Notice the agent is unable to consume this period. This rounds out the model.

VII. Results

The first set of results deal with the RBC model. The RBC model has 4 parameters: α, β, δ, and ρ as defined in the Economic models section. The CRRA elasticity, γ is set to 2. The model is simmulated for 200 iterations and the first 100 iterations is dropped to get a steady state behavior. The results of the posterior with the parameters displayed in the same order as above is shown here: The red bar indicates the true parameter values and the blue line indicates the posterior of the model as estimated by the simulation-based inference approach. Each of the density plots in the upper triangular portion of the graph indicates two way densities given by the corresponding row and column. Likewise the red dot indicates the true value of the parameter with respect to the

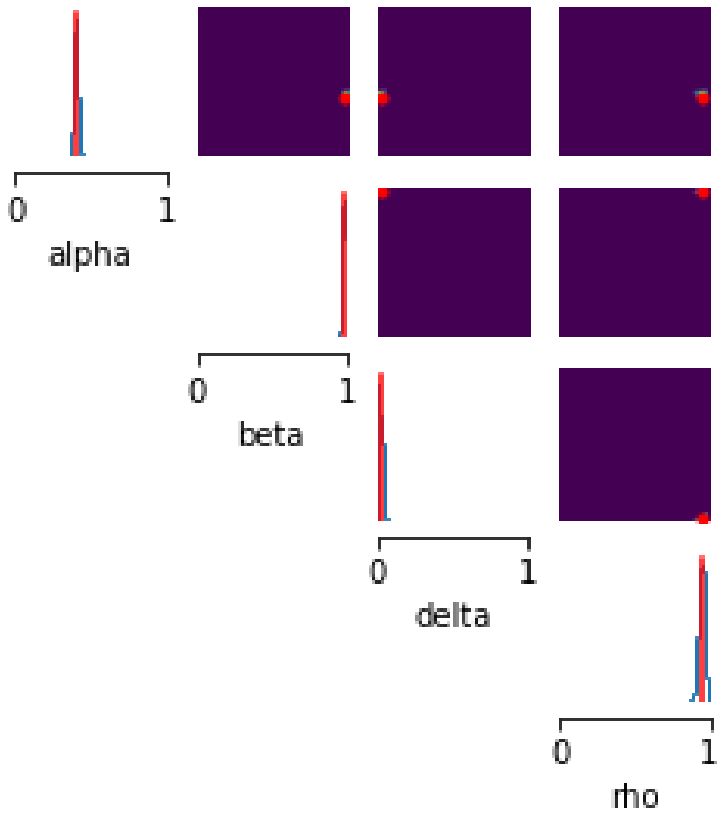

Figure II: Simulated Data RBC

two axis. Uniform priors are used along the 0-1 interval for all parameters.

As is clear, the simulation based inference approach concentrates almost entirely on the true parameter value. This extremely close relationship is not entirely replicated on the other models, as the other models are are more complicated and have flatter likelihoods in many areas. Additionally fewer time steps may be used in other models, making the posterior less able to concentrate around the true parameter values. That being said, this is powerful evidence that the simulation based inference approach can recover correct posteriors and likelihoods given only simulated data.

The next set of results deal with a partial equilibrium model of capital solved via value function iteration. The parameters here are *alpha*, *delta*, *sigma*, and *rho*. *sigma* is the standard deviation of the productivity law of motion. The interest rate is set at 5 percent and beta is set as 1/(1+r). Like in many other models, estimating beta is difficult and the model has many flat likelihoods and converges to values that don't make as much sense. Thus it is easier to just calibrate beta. The model was estimated with three different methods, each of which obtaining nearly the same posterior, giving me confedence the approach has converged to the right solution. The first chart will show the posterior for a simulation based inference approach that uses a normalizing flow and a feed forward network as the embedding network (converting the high dimensional x data into a lower dimensional conditioning variable). The second chart will show the same model and data but using a RNN embedding network. The third chart will show a density estimator that is a GAN instead of a normalizing flow.

The second chart is the same RBC model estimated on real data: Here there is no available ground truth. You can see that compared to the first chart, the method is estimating posteriors and not delta functions. You can also see the model doesn't estimate parameters close to what theory would suggest. This is not surprising given that the RBC model is to simple to work on real data and many of these parameters don't agree with theory even in more complex models. In the model, it predicts alpha should be very close to 1, which is reasonable since there is not labor growth in this model and capital is highly correlated with labor, which when combined usually implies a Cobbs-Douglas function that is homogeneous with degree 1.

Beta is also doesn't agree with theory, which is unsurprising as beta is often calibrated because

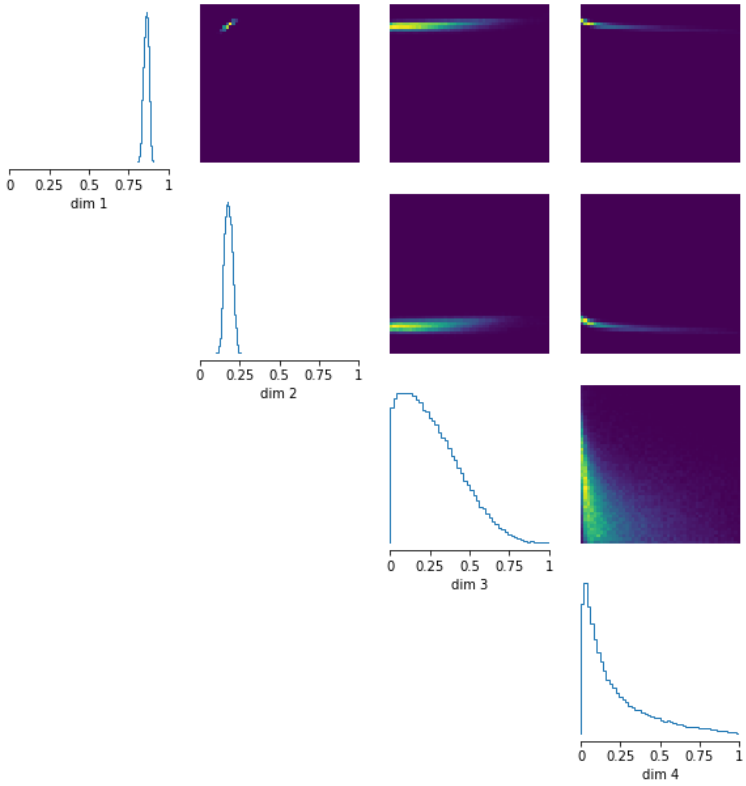

Figure III: Real Data RBC

it's so difficult to pin down. Likewise rho is another variable that is often estimated but is also difficult to pin down–likely because different parameterizations have an important but subtle effect on estimation. This is displayed by the relatively uncertain posterior. Delta or the depreciation rate, is the only parameter where the mode is relatively reasonable, but the posterior is fairly uncertain about it's value.

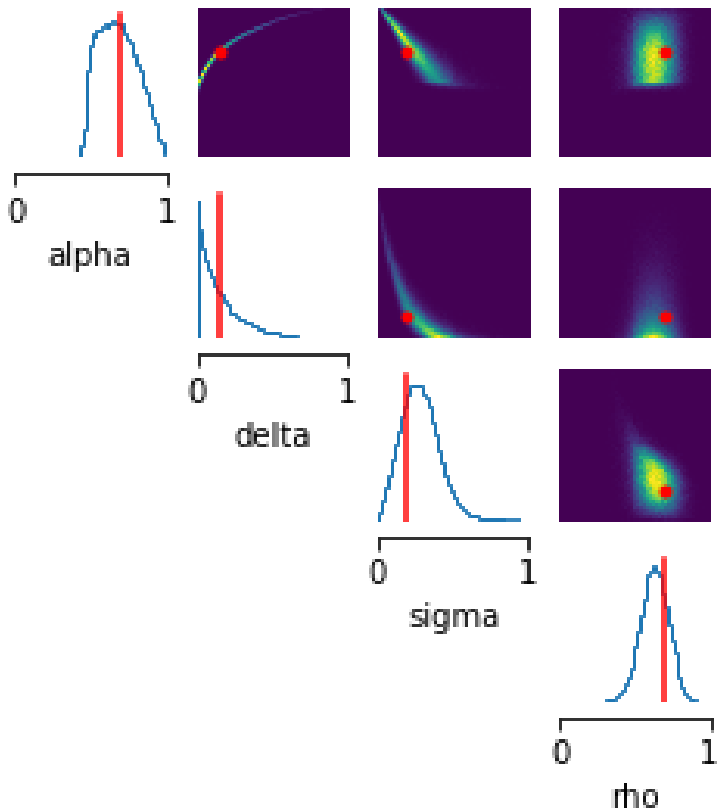

Figure IV: Value Function Iteration with a Dense Net Embedding

Desite using different inference techniques, all the posteriors look fairly equivalent, giving me confidence that the algorithm has converged and likely converged to the correct answer. Furthermore it does seem like the mode is fairly close to the actual parameterization, except for the depreciation rate which is much closer to the mode when you consider a three way interaction (not shown) between the first three dimensions. There some higher order considerations that are getting

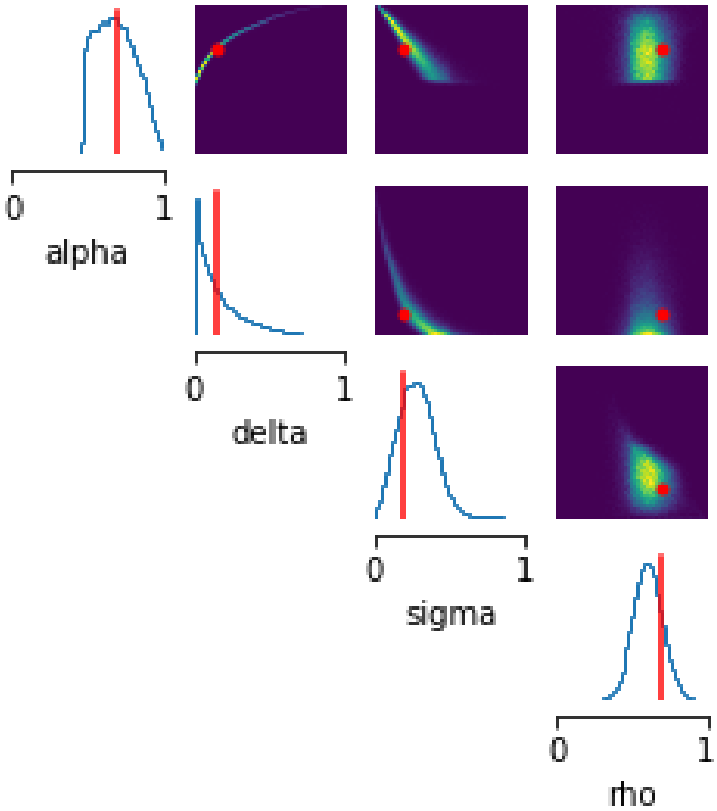

Figure V: Value Function Iteration with an RNN Embedding

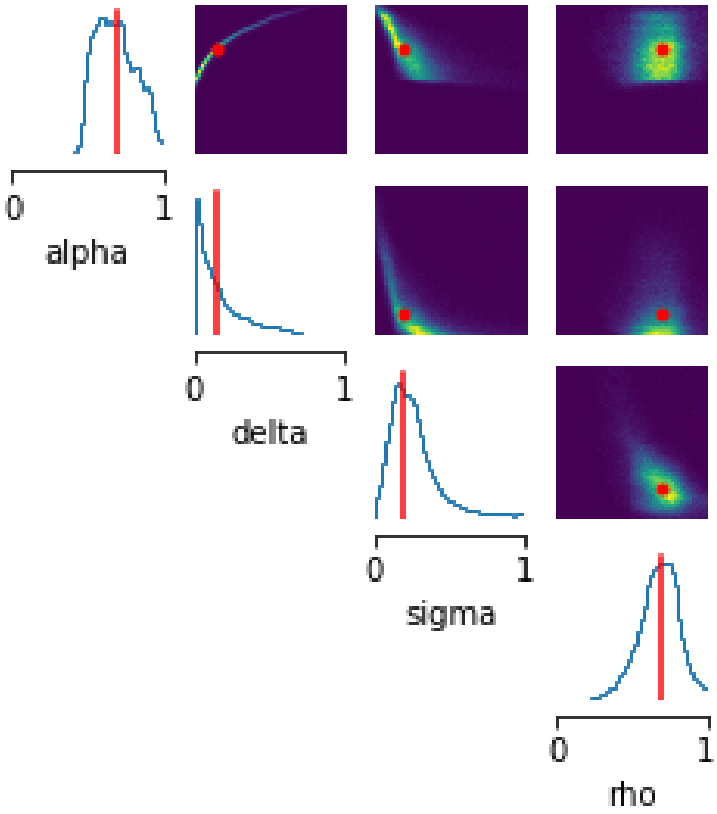

Figure VI: Value Function Iteration with a GAN Density Estimator

in the way of the dynamics of the model.

The third model is the 10 parameter HANK model with parameters (in order): need params.

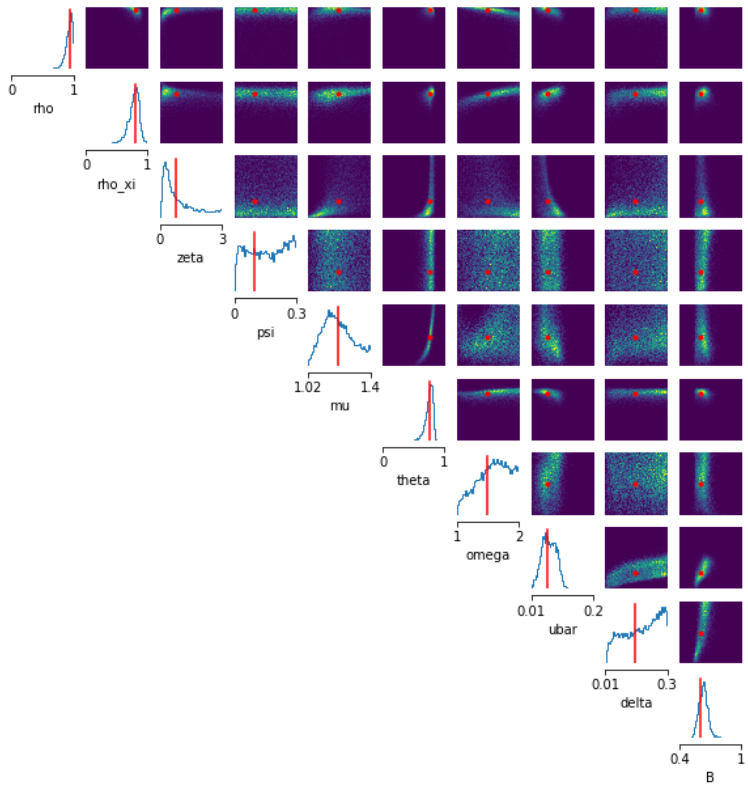

Figure VII: Ten Parameter HANK model

While potentially this model has not converged entirely due to computational issues, it's still impressive that the normalizing flow approach can estimate relatively formidable model especially one that has 61 timesteps and 829 covariates as many quantiles of the state distributions are covariates in the model.

The fourth model is the 11 parameter HANK model with parameters displayed in the chart

While many of the parameters have converged to their true value, some of the parameters may still have high uncertainty. This could be due to lack of fully convergence of the algorithm, or could be due to the fact that the parameters have very flat likelihoods. Regardless, this and the above model are certainly models are at the scale where real papers are being written with models of

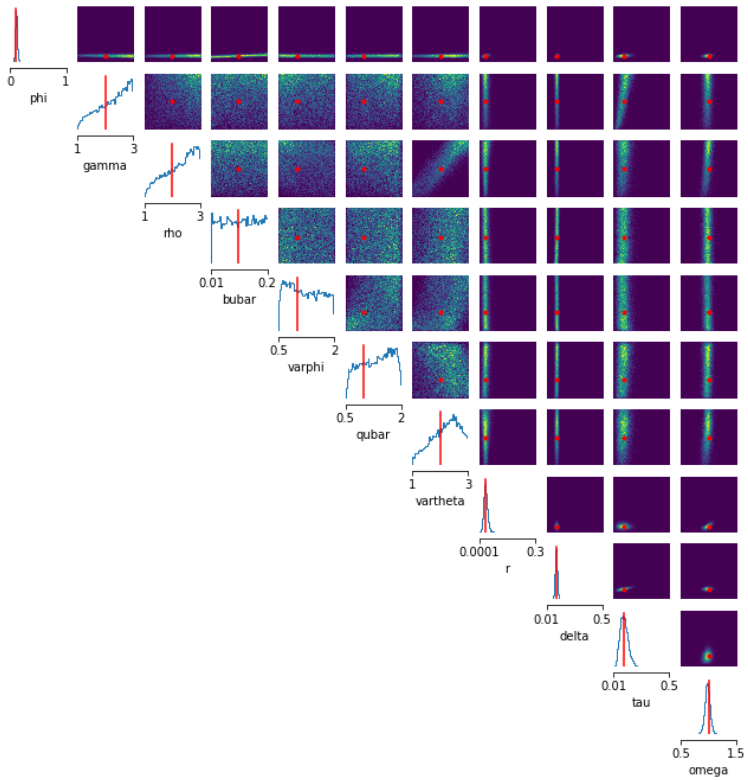

Figure VIII: Eleven Parameter Model Estimated via VFI

this size. Many of the posterior distributions do concentrate around the true value, suggesting the model does have some discerning power with regard to the given parameters. Furthermore this is a reasonable sized model solved via value function iteration and the model succeeds relatively well in calculating the posterior.

VIII. Conclusion

In this paper, I propose multiple methods to estimate models in a Bayesian or Maximum Likelihood manner using simulations only, without a likelihood function. This method works even on models that have a latent variable across time–the first such paper, to my knowledge, that does this in the most general manner in economics. This approach allows estimation of dynamic macro models like general value function iteration and projection solution methods in a Bayesian manner for the first time. As this approach is novel, there are still many applications in economics. Additionally, this approach to simulation based inference can be used to performing Bayesian inference in areas outside of macroeconomics like Industrial Organization, Game Theory, and Econometrics. The use of normalizing flows is fairly new to the field. The only application I know of in economics is the use to solve continuous time dynamic models (Childers, 2018), although it is not mentioned explicitly as a normalizing flow. The idea of a bijective map that can perform arbitrary change of variables is a very powerful technique that hasn't been explored adequately in economics, both as a proof technique and as an application. More to the aim of this paper, the use of this technique for Bayesian inference has many applications in macroeconomics, allowing for instance macroeconomists to estimate general models solved via value function iteration and projection with Bayesian Inference.

References

Altissimo, Filippo, and Antonio Mele. 2009. "Simulated non-parametric estimation of dynamic models." *The Review of Economic Studies*, 76(2): 413–450.

Barron, Andrew R. 1993. "Universal approximation bounds for superpositions of a sigmoidal function." *IEEE Transactions on Information theory*, 39(3): 930–945.

Brehmer, Johann. 2021. "Simulation-based inference in particle physics." *Nature Reviews Physics*, 3(5): 305–305.

Brownlees, CT, D Kristensen, and Y Shin. 2011. "Smooth filtering and likelihood inference in dynamic latent variables models." *Manuscript, Department of Economics, UCL.*

Carrasco, Marine, Mikhail Chernov, Jean-Pierre Florens, and Eric Ghysels. 2007. "Efficient estimation of general dynamic models with a continuum of moment conditions." *Journal of econometrics*, 140(2): 529–573.

Childers, David. 2018. "Solution of rational expectations models with function valued states." *Manuscript, Carnegie Mellon University.*

Chopin, Nicolas, Pierre E Jacob, and Omiros Papaspiliopoulos. 2013. "SMC2: an efficient algorithm for sequential analysis of state space models." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 75(3): 397–426.

Cranmer, Kyle, Johann Brehmer, and Gilles Louppe. 2020. "The frontier of simulation-based inference." *Proceedings of the National Academy of Sciences*, 117(48): 30055–30062.

Cranmer, Kyle, Juan Pavez, and Gilles Louppe. 2015. "Approximating likelihood ratios with calibrated discriminative classifiers." *arXiv preprint arXiv:1506.02169.*

Cunningham, Edmond, Renos Zabounidis, Abhinav Agrawal, Ina Fiterau, and Daniel Sheldon. 2020. "Normalizing Flows Across Dimensions." *arXiv preprint arXiv:2006.13070.*

Cybenko, George. 1989. "Approximation by superpositions of a sigmoidal function." *Mathematics of control, signals and systems*, 2(4): 303–314.

Dixit, Avinash K, and Joseph E Stiglitz. 1977. "Monopolistic competition and optimum product diversity." *The American economic review*, 67(3): 297–308.

Duffie, Darrell, and Kenneth J Singleton. 1990. "Simulated moments estimation of Markov models of asset prices."

Durkan, Conor, Iain Murray, and George Papamakarios. 2020. "On contrastive learning for likelihood-free inference." 2771–2781, PMLR.

Farrell, Max H, Tengyuan Liang, and Sanjog Misra. 2018. "Deep neural networks for estimation and inference: Application to causal effects and other semiparametric estimands." *arXiv preprint arXiv:1809.09953.*

Fernández-Villaverde, Jesús, and Juan F Rubio-Ramírez. 2007. "Estimating macroeconomic models: A likelihood approach." *The Review of Economic Studies*, 74(4): 1059–1087.

Fernández-Villaverde, Jesús, Juan Francisco Rubio-Ramírez, and Frank Schorfheide. 2016. "Solution and estimation methods for DSGE models." In *Handbook of macroeconomics.* Vol. 2, 527–724. Elsevier.

Flury, Thomas, and Neil Shephard. 2011. "Bayesian inference based only on simulated likelihood: particle filter analysis of dynamic economic models." *Econometric Theory*, 27(5): 933–956.

Gallant, A Ronald, and George Tauchen. 1996. "Which moments to match?" *Econometric theory*, 12(4): 657–681.

Goodfellow, Ian, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. "Generative adversarial nets." *Advances in neural information processing systems*, 27.

Gourieroux, Christian, Alain Monfort, and Eric Renault. 1993. "Indirect inference." *Journal of applied econometrics*, 8(S1): S85–S118.

Greenberg, David, Marcel Nonnenmacher, and Jakob Macke. 2019. "Automatic posterior transformation for likelihood-free inference." 2404–2414, PMLR.

Haan, Peter, and Arne Uhlendorff. 2006. "Estimation of multinomial logit models with unobserved heterogeneity using maximum simulated likelihood." *The Stata Journal*, 6(2): 229–245.

Hajivassiliou, Vassilis A, et al. 1997. *Some practical issues in maximum simulated likelihood.* Suntory-Toyota International Centre for Economics and Related Disciplines ....

Hennessy, Christopher A, and Toni M Whited. 2007. "How costly is external financing? Evidence from a structural estimation." *The Journal of Finance*, 62(4): 1705–1745.

Herbst, Edward, and Frank Schorfheide. 2014. "Sequential Monte Carlo sampling for DSGE models." *Journal of Applied Econometrics*, 29(7): 1073–1098.

Hermans, Joeri, Volodimir Begy, and Gilles Louppe. 2020. "Likelihood-free mcmc with amortized approximate ratio estimators." 4239–4248, PMLR.

Huang, Chin-Wei, David Krueger, Alexandre Lacoste, and Aaron Courville. 2018. "Neural autoregressive flows." 2078–2087, PMLR.

Hyvärinen, Aapo, and Petteri Pajunen. 1999. "Nonlinear independent component analysis: Existence and uniqueness results." *Neural networks*, 12(3): 429–439.

Judd, Kenneth L, Lilia Maliar, Serguei Maliar, and Inna Tsener. 2017. "How to solve dynamic stochastic models computing expectations just once." *Quantitative Economics*, 8(3): 851–893.

Kaji, Tetsuya, Elena Manresa, and Guillaume Pouliot. 2020. "An adversarial approach to structural estimation." *arXiv preprint arXiv:2007.06169.*

Keane, Michael P, and Kenneth I Wolpin. 1994. "The solution and estimation of discrete choice dynamic programming models by simulation and interpolation: Monte Carlo evidence." *the Review of economics and statistics*, 648–672.

Kingma, Diederik P, and Max Welling. 2013. "Auto-encoding variational bayes." *arXiv preprint arXiv:1312.6114.*

Kristensen, Dennis, and Yongseok Shin. 2012. "Estimation of dynamic models with nonparametric simulated maximum likelihood." *Journal of Econometrics*, 167(1): 76–94.

Kukackaa, Jiri, and Jozef Barunika. n.d.. "Non-Parametric Simulated Maximum Likelihood Estimation of Parameters in Heterogeneous Agent ModelsI Preliminary Draft Version: May 19, 2015."

Lerman, Steven, and Charles Manski. 1981. "On the use of simulated frequencies to approximate choice probabilities." *Structural analysis of discrete data with econometric applications*, 10: 305–319.

Li, Yujia, Kevin Swersky, and Rich Zemel. 2015. "Generative moment matching networks." 1718–1727, PMLR.

McFadden, Daniel. 1989. "A method of simulated moments for estimation of discrete response models without numerical integration." *Econometrica: Journal of the Econometric Society*, 995–1026.

Pakes, Ariel, and David Pollard. 1989. "Simulation and the asymptotics of optimization estimators." *Econometrica: Journal of the Econometric Society*, 1027–1057.

Papamakarios, George. 2019. "Neural density estimation and likelihood-free inference." *arXiv preprint arXiv:1910.13233*.

Papamakarios, George, David Sterratt, and Iain Murray. 2019. "Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows." 837–848, PMLR.

Reiter, Michael. 2009. "Solving heterogeneous-agent models by projection and perturbation." *Journal of Economic Dynamics and Control*, 33(3): 649–665.

Rendahl, Pontus. 2017. "Linear time iteration." IHS Economics Series.

Rezende, Danilo, and Shakir Mohamed. 2015. "Variational inference with normalizing flows." 1530–1538, PMLR.

Rubin, Donald B. 1984. "Bayesianly justifiable and relevant frequency calculations for the applied statistician." *The Annals of Statistics*, 1151–1172.

Smith Jr, Anthony A. 1993. "Estimating nonlinear time-series models using simulated vector autoregressions." *Journal of Applied Econometrics*, 8(S1): S63–S84.

Tran, Dustin, Rajesh Ranganath, and David M Blei. 2017. "Hierarchical implicit models and likelihood-free variational inference." *arXiv preprint arXiv:1702.08896*.

IX. Appendix

A Proofs

B Outline

I will first write a basic outline of the estimation procedure, so that you will have a road map of how the estimation works before working through proof that this procedure works. There are two main parts of the methodology: 1.) How the Bayesian estimation procedure works, 2.) The estimation procedure requires the use of a normalizing flow density estimator, and I will prove and discuss properties of that model

C Bayesian Estimation Outline

At its most basic the procedure takes samples of the parameters from the prior, samples fake data from the likelihood (conditional on the prior), and concatenates the parameters with the fake data to generate samples from the joint distribution of the model. Knowing this one can estimate the posterior by fitting a conditional density estimator $P(\theta|x)$ on the joint samples. This is achieved via a normalizing flow density estimator.

D Normalizing Flow Outline

A normalizing flow is a stacking of an arbitrary number of neural network architectures called bijectors. A bijector, like the name suggests, maps one distribution to another in a bijective way. In the forward direction you sample from one distribution (called the base distribution) put the sampled values into the bijector to get an output. Likewise, you can sample the other distribution (called the target distribution) put the samples of the target distribution into the output of the bijector and run the bijector in the inverse direction to get an associated input. Like the name suggests, you can recover respective samples of the other distribution by sampling from the original distribution and running through the bijector in the correct direction. Likewise, bijectors are (weakly) differentiable, so that the relationship between the two distributions are diffeomorphic,

preserving measure, thus one can not only recover samples given one distribution, but probability measure given the probability measure of the other distribution, using a change of variables formula.

E Bayesian Estimation In Depth

The first step is to draw samples from the joint distribution and I will show this is possible. Since $P(X, \theta) = P(X|\theta)P(\theta)$, the best procedure is to draw θ from the prior, then draw X from the likelihood $P(X|\theta)$. This draws one sample (X, θ) from the joint distribution. Since the posterior is defined as $P(\theta|X)$, if one can fit a conditional density estimator on $P(\theta|X)$ on the joint samples (X, θ) one can get the posterior distribution of theta. One can then condition on the true data X* to get the distribution of theta with respect to the actual data that was seen. Now I haven't mentioned a good density estimator that can do this as kernel density estimators will not work well but I will discuss the normalizing flow density estimator later, but for now take that as a given.

This procedure is enough to generate posterior distributions. However, this can be computationally slow (but is still often used as it works fine) if the prior is far away from the posterior as you are sampling theta in places where the posterior has little mass. A more computationally efficient way is to sample in multiple rounds. In the first round one uses the prior distribution, however in later rounds one samples $P(\theta)$ from a proposal distribution which should be the current most updated guess on the posterior (denoted with a hat) $\hat{P}(\theta|X)$, then one adjusts via importance sampling for the fact that $\hat{P}(\theta|X)$ is not the same as $P(\theta)$. In order to perform the adjustments note that $P(\theta|X) = P(X|\theta)P(\theta)/P(X)$. Call the incorrect proposal distribution $\hat{P}(\theta)$ and define $\hat{P}(\theta|X) = P(X|\theta)\hat{P}(\theta)/P(X)$. Then

$$\hat{P}(\theta|X) = (P(X|\theta)P(\theta)/P(X)) * (\hat{P}(\theta)/P(\theta)) = P(\theta|X) * (\hat{P}(\theta)/P(\theta))(eq1)$$

Since the joint distribution with the differing proposal distribution has a pdf of $P(X|\theta)\hat{P}(\theta)$ so the conditional distribution which one is drawing samples from has pdf $\hat{P}(\theta|X)$. If one is fitting a density estimator $Q(\theta|X)$ on the differing proposal joint data via maximum likelihood as one does with a normalizing flow, one will recover estimates the distribution $\hat{P}(\theta|X)$. However, if one fits the

density estimator $Q(\theta|X) * (\hat{P}(\theta)/P(\theta))$ (ie adjusting the likelihood by the importance weights), then $Q(\theta|X)$ will be the residual after accounting for the factor $(\hat{P}(\theta)/P(\theta))$, which by eq 1 is $P(\theta|X)$.

However, this requires that both the prior and the posterior can be both easily sampled from as well as easily calculable density. A normalizing flow can do this as typically you choose a base density that has this property, and by the fact that the normalizing flow is a stack of bijections with tractable change of variable calculations, you can calculate the pdf and sample from the estimate of the target distribution. In multi round simulation-based inference, the prior is the first proposal distribution $\hat{P}(\theta)$ and the normalizing flow is the resulting first round estimate of the posterior: $Q_1(\theta|X)$. Then for the second round $\hat{P}(\theta) = Q_1(\theta|X = X*)$, where the proposal is now the estimated posterior evaluated at the real data recovered in the first round. Later rounds iteratively replace the proposal $\hat{P}(\theta) = Q_{n-1}$ with the estimated posterior recovered from the previous round (and importance sample). Because importance sampling requires the ratio: $(\hat{P}(\theta)/P(\theta))$, Q needs to be both easily sampled from as well as having easily calculable pdf which a normalizing flow does have. I have demonstrated how to perform multi round inference in a Bayesian fashion assuming the existence of a normalizing flow-like density estimator.

F Normalizing Flows in Depth

A normalizing flow is a complex model. I will discuss the constituent parts. Here is a decent description of them on an intuitive level: `http://akosiorek.github.io/ml/2018/04/03/norm_flows.html`. I encourage reading the blog post to understand flows at an intuitive level. I will be trying to be somewhat technical and proving relationships to some degree, so understanding the intuition behind flows may be difficult by reading this.

The point of a normalizing flow is to estimate the density from samples. The idea is to learn a mapping from an easy to model distribution like the multivariate normal distribution, to the target distribution which is learned via samples. The way this is done take sample points from this target distribution and take the inverse bijection mapping and get the respective base space points. Since the density of the base space distribution is known, using a change of variable formula, one can

calculate the density (or likelihood) the flow assigns to any sample point in the target space. Then one optimizes the parameters of the flow via maximize likelihood.

Before starting I will prove a couple lemmas:

lemma a: If a set E in X has zero measure and a function $g(X)$ is Lipschitz, then $g(E)$ will also have zero measure:

Suppose $E \subset A$ has measure zero. Then cover A with a countably finite union of increasing compact sets in R^n such that:

$$A \subset \bigcup_{k=1}^{\infty} C_k.$$

Let E_k denote the intersection of E with C_k. Define a measure on R^n in the target space μ. If $\mu(g(E_k)) = 0$ for all k then $\mu(g(E)) = 0$ as a countable union of measure zero sets is measure zero. Since E_k has measure zero, for any $\delta > 0$ there is a cover of E_k with open balls $\{B(x_i, r_i)\}_{i=1}^{m}$ such that their total measure is less than or equal to δ. So,

$$E_k \subset \bigcup_{i=1}^{m} B(x_i, r_i).$$

If there is an $x \in E_k$ there one of the m balls that covers E_k covers x: $|x - x_i| \leq r_i$. Thus if a function is lipshitz $|y - g(x_i)| \leq \lambda_k r_i$ where λ_k is the limpshitz constant. Thus

$$\mu(g(E_k)) \leq \mu\left(\bigcup_{i=1}^{m} B(g(x_i), \lambda_k r_i)\right) \leq \lambda_k^n \delta.$$

As the radius of the balls go to zero as δ goes to zero and the entire set is still covered. Thus $\mu(g(E_k)) = 0$, proving $\mu(g(E)) = 0$

QED

lemma b: If a function J maps a random variable (the base distribution) X to another random variable Y (the target), and both random variables are subset of R^n, an a sequences of functions J_n converges to J pointwise, then $Y_n = J_n(X)$ converges in distribution to $Y = J(X)$.

Proof (from (Huang et al., 2018))

Let h be any bounded continuous function on R^n. h composed with J_n converges pointwise to

$h \circ J$ as compositions of continuous functions converge to their compositions. Since h is bounded, by the dominated convergence theorem $E[h(Y_n)] = E[h(J_n(X))] \to E[h(J(X))] = E[h(Y)]$. Since this result holds for any continuous function by the Portmanteau theorem this implies $Y_n \to Y$.

QED

Lemma c: $y_i = w(x_1, ...x_{i-1}) * \sigma(\alpha(x_1, ...x_{i-1}) * x_i + \beta(x_1, ...x_{i-1}))$ with α, β, and w being outputs from neural networks with a postive output either via postive weights or a positive nonlinearity (on α and β) and a monotonic non-linearity on the over all flow, σ, is monotonically increasing with each bijector.

To prove that a bijector parametrized in this way is monotonic, take a derivative of y_i with respect to x_i.

$$dy_i/dx_i = d\sigma(a)/da * da/dx_i$$

Where a corresponds to the equation inside of σ. Since $\sigma(.)$ is monotonic, $d\sigma(a)/da$ has a derivative greater than 0. Likewise $da/dx_i = \alpha(x_1, \ldots x_{i-1})$ which by assumption is set to be positive either with a positive non-linearity or a correct composition of weight and domain restrictions. Thus each dimension of the bijector is monotonic making the entire bijector monotonic (increasing). A composition of monotonically increasing functions is monotonically increasing so the entire normalizing flow is monotonically increasing via induction.

QED

Now that these lemmas have been proven, I will discuss an individual bijector. I will first show why it is bijective in event space (ie samples) and then move to probability space and show why it also is a valid change in measure and how to calculate the change of variables so that if one knows the pdf of the base distribution, one also knows the distribution of the target distribution and vice versa. Lastly, I will discuss that if the model is set up in a particular way, it should be able to universally approximate any smooth bijective mapping between two random variables, and thus should be able to map any continuous base distribution to any continuous target distribution, which allows this density estimator to have "non-parametric" like properties.

While there are many types of bijectors, the one I use and the most common one is the au-

toregressive bijector. At this point, I will work in event space, so its not important to think about probability distributions but rather a mapping between two sets of numbers in R^n (ie samples from a multivariate distribution). This is a bijection so the dimensionality between samples in the base space and the target space should be the same. Call it n (this can be relaxed but it's not important in this paper). The autoregressive flow orders the input and the output as so $X = \{x_1, x_2 \ldots x_n\}, Y = \{y_1, y_2 \ldots y_n\}$. The name autoregressive comes from the fact that each output variable is only nonlinearly conditioned on input variables with a smaller index and y_1 is just the identity of x_1. Ie $y_5(x_1, x_2, x_3, x_4), y_3(x_1, x_2), y_1 = x_1$. The reason this is done is to allow for bijections. Clearly if you know what all the x values are you will know all the y values via the forward equations $y_i(x_1..x_{i-1})$. However, if knowing all the y_i's allows recovering all the x_i's as well. This is because $y_1 = x_1$ then y_2 is only a function of x_1 so by inverting y_2, one can get x_2. Then one knows all the conditioning variables for the $y_3(x_1, x_2)$ equation and if one can invert that one can get x_3 etc.

In order to construct y_i functions that are both expressive and also invertible knowing the conditioning variables, notice that if I have a function $y_i = \sum_k w_k(x_1, ...x_{i-1}) * \sigma(\alpha_k(x_1, ...x_{i-1}) * x_i + \beta_k(x_1, ...x_{i-1}))$, if I know all the conditioning x_i terms and the forward functions α and β, no matter how complex α and β are, I can get x_i from y_i and y_i from x_i. This is how an autoregressive flow is constructed. Each y_i is a linear function in α and β, along with an invertible nonlinearity, σ, like a leaky rectified linear unit as well as another weight w. Because $\alpha(x_1, ...x_{i-1}) * x_i + \beta(x_1, ...x_{i-1})$ is affine in x, it is clear if you know the x's you can recover the input into σ. σ is also a bijective nonlinearity, so $\sigma(.)$ is also a bijection. Then multiplying by another constant is also a bijection.

After laying down the intuition, it is time to prove a bijector is a bijection.

Theorem 1: A bijector learns a bijection between the input and output space.

Proof:

This will be done by proving three different elements are bijections and then the k sum of these elements are bijections. First $y' = \alpha(x_1, ...x_{i-1}) * x_i + \beta(x_1, ...x_{i-1})$ is a bijection. Then $\sigma(y')$ is a bijection and finally $y = w(x_1, ...x_{i-1}) * \sigma$ is one. In order to prove that the first relationship is onto, first recognize the identity function is onto. Thus given any i, α and β are fixed as a

function of earlier x's, thus given a value for $y_i = y$, the x in the preimage that generates y can be recovered with this equation $(y_i - \beta)/\alpha$, thus for any given y_i one can recover a x_i proving $x \to y(x)$ is onto, then by induction the multiavariate function is onto. To prove that this function is 1-1 proceeds in the same way. The identity is 1-1 and if $y_i(x_i) = y_i'(x_i')$, then $x_i = x_i'$, again by the fact that affine functions are 1-1 and one extends this to all dimensions with induction. Proving $\sigma(y')$ is a bijection is by definiton. σ is choosen to be a bijective nonlinearity. Proving the final step, $y = w(x_1, ...x_{i-1}) * \sigma$, as this again is linear function in σ which is a subset of affine functions and in the first set of equations, I proved that affine functions are bijections. The final step involves proving the bijectiveness of y_i is made up of a sum of k of these equations. However, it is not true that a sum of bijections is a bijections. For example take $f = x$ and $g = -x$. One condition for a sum of bijections to be a bijection is for the bijections to be monotonic. This is exactly the condition that is necessary to perserve the orientation of the underlying space that I talk about below. In lemma c, I prove that if the weights on the neural network in a neural autoregressive bijector are all positive, this will make the model monotic. Given two monotonic continuous bijectors f and g, the sum is onto. Define h as $f + g$. h is continuous as a sum of continuous functions is continuous. Given a value y in the interior of the range of h there exists a value of $h|_i > y$ and a value of $h|_j < y$. Since h is continuous, by the intermediate value theorem it takes on a value of y. Likewise to prove 1-1, h is also monotonic as a sum of monotonic functions is monotonic. Thus, it can only take on that value of y exactly once. One can extend the proof of two monotonic bijections to arbitrary monotonic bijections by using induction. Combining the neural autoregressive bijector with a bijector that permutes the indices of x_i (which allows conditioning each variable ultimately on all other variables when bijectors are stacked together). Now once you have a stack of bijectors, where the output of one bijector is the input of another, this is also a bijector as a composition of bijections is a bijections.

QED

Proving bijectiveness, now I will work in the underlying density space or the space of measures and move away from just viewing the flow as a mapping of realizations of random variables. Not only is this a bijection, but because you can differentiate the bijector with respect to x and y

and x and y are smooth random variables then this is a diffeomorphism (at least weakly). If the function is weakly diffeomorphic or even if the derivatives with respect to the input and outputs are bounded, then there exists a constant C such that the absolute value of the derivative over in any compact set in R^n is bounded by C. Define the compact set that includes the data to be any set that includes all sample from the data generating process with probability $1 - \delta$. The boundedness of the derivative inside a given compact domain containing the data must be true for the bijector because the with respect to x or y is just a product of all the $\alpha(x_1, \ldots x_{i-1})$'s that multiply the given input that are also shifted by β. These are neural networks and so the product is bounded by a constant $(\max_{|\alpha|,|\beta|} |\alpha| + |\beta|)^N$, $\max_{\alpha,\beta} \alpha + \beta$ is the largest value of any α and β neural network takes when taking in any data in this set and replace beta with 1 if β-max is less than one. N then is just the number of layers. Likewise the reverse direction finds the max for the inverse values for alpha. Since the derivative is bounded this implies that the function is C-Lipschitz over any particular compact set that includes our data up to probability δ, as the C constant can be derived by integrating over the constant bounding the derivative.

Theorem 2: The chain of neural autoregressive bijectors along with a bijector that permutes the indices of inputs, learns a valid change in measure of a input base distribution to an output target distribution and the pdf of the target distribution can be derived using the standard change of variable formula: $q(y) = q(x)|det(df/dx)|^{-1}$

I will now use lemma a proved earlier that any Lipschitz functions preserves sets with measure zero. This implies that measures of both spaces are absolutely continuous with respect to one another. Thus in both directions there exist a Radon Nikodym derivative. The Radon Nikodym derivative of the transformation of one random variable transformed by a bijector follows the convenient change of variables formula: $q(y) = q(x)|det(df^{-1}/dx)|$ evaluated at a point x. where f maps x to y $(f(x) = y)$. This can be even more conveniently expressed as: $q(x)|det(df^{-1}/dx)| = q(x)|det(df/dx)|^{-1}$ This relationship can be shown by using the inverse function theorem and the recognition that the determinant of an inverse is the inverse of the determinant: $|det(df^{-1}/dx)| = |det((df/dx)^{-1})| = |det((df/dx))^{-1}| = |det(df/dx)|^{-1}$, with derivatives all evaluated at a point x. Implicit in all this is lemma b which proves that if a sequence of flows f_n converges to $Y = f(x)$

pointwise, then $Y_n \to Y$.

Now I have shown given a sample from y and a given distribution like x, how to recover the pdf a bijector assigns to the sample point y. First one inverts the bijector to get x. Then one uses the change of variables formula to get the pdf of y under the bijector. One then can optimize the bijector using maximum likelihood using q(y) as the likelihood of y.

A normalizing flow is just a stack of bijectors on top of one another so the output of one bijector becomes the input of the next bijector. Composition of bijections are also bijections, this flow is invertible. Likewise a composition of diffeomorphisms is still diffeomorphic and so this flow also has a valid change of measure. However when composing flows we are interested in peserving some sort of orientation. For example, we would like a sample from the base distribution that has a larger value along every dimension of the sample space to also correspond to be mapped to a sample in the target space that is also larger along every dimension. If each individual bijector is not monotonicly increasing in every dimension this may not hold, as for example multiplying one dimension by a negative number for certain values would flip one axis conditionally while leaving the rest of the axes the same which may result in the CDF of some value in the base space corresponding to a non-CDF set of patches in the target space. I will not prove this effect with non-monotonic flows. However, it is clear that if each bijector is monotonely increasing in all dimensions than the entire flow is monotonicly increasing in all dimensions, preserving orientation of the underlying space (as the determinant of such a transformation is always positive). Thus a flow has a likelihood that can be calculated with this formula: $q(y) = q(x)\pi_{i=1}^{n}|det(df_i/dx)|^{-1}$ and the determinant is never negative.

QED

Finally I will show that a certain construction of normalizing flows are universal approximators of multivariate real probability distributions. I focus on proving this universal approximator properties for a single layer (but arbitrary width) normalizing flow.

Since the variables are conditioned only on previously indexed variables, of course a single bijector cannot approximate any dependency relationships between a target variable and a base variable if the index of the particular dimension of the base variable is larger than the index

of the particular dimension of the target variable. In practice this can be avoided by stacking multiple bijectors and permuting the indices of the input. As I will show, if a bijector can universal approximate any distribution conditional on all base variables with lower indices, then composing two bijectors where the first order of variables is arbitrary and the second order of variables is the reverse order of the first variable, this stack of bijectors will have variables that are conditioned on all input variables and combined with universal approximator properties of any bijector layer, this multiplayer bijector will approximate any distribution with arbitrary conditioning. Thus, any conditioned relationships not in the first bijector will be in the second. This will result in a flow that can approximate any distribution with arbitrary conditioning relationship, as both flows are universal approximators given the allowed conditioning structure and combined they can model arbitrary conditioning structure.

G Normalizing Flows are Universal Approximators

The proof that normalizing flows parameterized in this way are universal approximators will proceed in a series of steps following Huang et al. (2018). First, any continuous probability distribution can be mapped to any other probability distribution with a monotonic function. This will be made formal by showing monotonic functions and can map any pdf to and from the multivariate uniform distribution. Before that can be shown, I will show that the flow can approximate any monotonoic function. I will first show that a sum of step functions are universal approximators of monotonic functions. Then I will show that if we make the nonlinearity σ a logit transformation, one can modify the logit transform to be arbitrarily close to a given step function. Likewise a similar argument holds for something like a hyperbolic tangent nonlinearity and likely some proof can be constructed using a leaky RELU along with a negative leavy RELU. I will stick to proving it for a logit transformation. Finally the proof will show that a neural network that approximates the optimal parameters w, α, β of the logit function can universally approximate any monotonic function. Finally I will show that there exists monotonic transformations that can map any distribution to any other via the uniform distribution.

In lemma 1-3 the domain and range of the two functions are between two arbitrarly constants

(r_0, r_1)

Lemma 1: Step functions universally approximate monotonic functions.

Define:

$$S*(x) = \sum_k w_k * s(x - b_j)$$

Where s is the step function that steps at $x = b_j$

Then the lemma states that for any continuous and monotonically increasing function $S : [a, b] \to [0, 1]$ and given any $\epsilon > 0$ there exists an $S*(n)$ for some n such that $|S*(x) - S(x)| < \epsilon \forall x \in [a, b]$. Abbreviating $s_j(x) = s(x - b_j)$, choose an $n = \lceil 1/\epsilon \rceil$ and devide the range $[0, 1]$ into n+1 evenly spaced intervals: $[0, y_1), [y_1, y_2)...[y_n, 1]$. For each y_j, there is a corresponding x_j value that corresponds to $x_j = S*^{-1}(y_j)$, since $S*$ is strictly monotonic. Now find the step function so all the $S*(x_j) = y_j$, which will mean the error is at most ϵ as the difference in y_j is at most ϵ. To do this ensure that $s_j(x)$ steps at x_j by setting $b_j = x_j$. Then set the weights w_j so that w_j makes up the difference between y_{j-1} and y_j. This is equivelent to solving $Sw = t$ where S is the lower triangular matrix of ones that represents which steps have activated and t is the vector of $[0, y_1, y_2...y_n, 1]$. The distance between $t_j - t_{j-1} = 1/(n + 1)$, thus:

(50)

$$|S*(x) - S(x)| = |\sum_j s_j(x)(t_j - t_{j-1}) - S(x)|// \quad = |1/(n+1) * \sum_j s_j(x) - S(x)|// \leq 1/(n+1) < \epsilon$$

QED

Lemma 2: I will move on to proving the same thing with sigmoid functions of the form:

$$S'(x) = \sum_k w_k * \sigma((x - b_k)/\tau_k)$$

Also can universally approximate arbitrary monotonic functions.

Note this can easily be translated into a form similar to our bijector by setting $\alpha_k = 1/\tau_k$, $beta_k = b_k/\tau_k$, and $w_k = w_k$. Note at this point α, β and w are constants and not functions which

I will deal with in the next section. For any monotonic function, approximate the function using a step function up to a factor of $\epsilon/3$, using the previous lemma. Then the error of each gap is at most $\epsilon/3$. Define $S'(x)$ such that $w_k = w_j$ and $b_k = b_j$ where the j index refers to the step function parameters ($S*$) and the k index refers to the parameters in the sum of sigmoids (S'). Then choose $\tau = \tau_k = \dfrac{\kappa}{\sigma^{-1}(1 - 1/(n-1))}$ where $\kappa = \min_{l \neq l'} b_l - b_{l'}$. Then $|\sigma(x - b_k/\tau) - s(x - b_k)|$ is at most .5 if $b_{k-1} < x < b_{k+1}$ as the logit is around .5 right at b_k where the step steps from 0 to 1 and when the steps are from 0 to $\epsilon/3$, the error is at most $.5 * \epsilon/3$. However, if x is not in this range then $sigma(x - b_k/\tau) = \sigma(x - b_k/(\min_{l \neq l'} b_l - b_{l'}) * \sigma^{-1}(1 - 1/(n-1))$ which implies that the distance from the step is at most $1/(n-1)$ for a 0 to 1 step function. Putting this together you have $\epsilon/3$ from the sum of the two one halves, then $\epsilon/3 = (n-1)/(n-1) * \epsilon/3$ from each other sigmoid that is not within $b_{k-1} < x < b_{k+1}$, which gives $2/3\epsilon$ from the logit approximation to the step function and $\epsilon/3$ from the step function approximation to the monotonic function. Using the triangle inequality: $|S'(x) - S(x)| < |S'(x) - S*(x)| + |S*(x) - S(x)| < 2/3\epsilon + \epsilon/3 = \epsilon$

QED

Lemma 3: Let the base and target space be multivariate and α, β, and w now be the outputs of neural networks conditioned on previous data. Bijectors parameterized like this:

$$y'_i = \sum_k w_k(x_1, ...x_{i-1}) * \sigma(\alpha_k(x_1, ...x_{i-1}) * x_i + \beta_k(x_1, ...x_{i-1})) \tag{51}$$

universally approximates monotonic functions. I will prove this theorem by writing the bijectors in this form:

$$y'_i = \sum_k w_k(x_1, ...x_{i-1}) * \sigma(\frac{x_i - b_k(x_1, ...x_{i-1})}{\tau_k(x_1, ...x_{i-1})}) \tag{52}$$

Since i indexes all the outputs of the bijector, I will proceed by proving the theorem for univariate case for each $y'_i(x_1...x_i)$ and then extending the theorem for the multivariate function (eg the distance all i have from a multivariate function is smaller than a given ϵ) is relatively easy.

Proving this theorem requires showing there is a sequence of functions $y'_i(x_1..x_i)$ such there exists

a $k < K$, which refers to the number of weighted logits being summed in y'_i and an $n < N$, which refers to the number of parameters in the neural network generating $w_k(x_1, ...x_{i-1})$, $b_k(x_1, ...x_{i-1})$, and $\tau_k(x_1, ...x_{i-1})$, the error $|y'_i(x_1...x_i) - S(x_i)|$ is smaller than any $\epsilon > 0$. Here y' is the bijector and S is the monotonic function that needs to be approximated.

The proof relies on the well known universal approximator result of neural networks (Cybenko, 1989), which states that a wide enough neural network can approximate any multivariate function. Because this is true, there will be a sequence of $w_k(x_1, ...x_{i-1})$, $b_k(x_1, ...x_{i-1})$, and $\tau_k(x_1, ...x_{i-1})$ when one increases n such that these outputs approximates w_k, b_k, and τ_k, the fixed parameters in lemma 2 arbitrarily well.

Since the fixed parameters can approximate any monotonic function arbitrarily well, I have:

$$|S'(x_i) - S(x_i)| < \epsilon/2$$

I also know from Cybenko (1989), that a wide enough neural network can approximate any function. Choosing a function that matches the parameters $S'(x_i)$ with low enough error, I know there exists a neural network for the parameters $\alpha_k(.)$, $w_k(.)$, and $\tau_k(.)$ in y_i such that:

$$|y_i(x_1...x_i) - S'(x_i)| < \epsilon/2$$

Now this distance bound is only true because the function y_i is a bounded and continuous and so you can use the dominated convergence theorem to turn the the convergence of the sequence of parameters (e.g. $\lim_{n\to\infty} \alpha_k^n(.) \to \alpha_k$ etc), to a statement about the convergence of a sequence of functions: $\lim_{n\to\infty} y_i^n \to S'(x_i)$

Then the proof in the univariate case is finished by using the triangle inequality:

$$|y_i(x_1...x_i) - S(x_i)| < |y_i(x_1...x_i) - S'(x_i)| + |S'(x_i) - S(x_i)|// \qquad = \epsilon/2 + \epsilon/2 = \epsilon \tag{53}$$

The proof in the multivariate case is completed by recognizing that for each univariate case there is a K_i such that when $k_i > K_i$, $|y_i(x_1...x_i) - S(x_i)| < \epsilon$. Then just choose $k = \max k_i$ and

then $||y_i - S||_\infty < \epsilon$

QED

Now that I've demonstrated that a single very wide bijector is enough to approximate any monotonic function, it's clear stacking many wide bijectors can only improve model flexibility for a given width k. What is left to show is that monotone functions are enough to allow a normalizing flow to map any continuous base distribution to any target continuous distribution. The proofs will show that any distribution can be mapped to a multivariate uniform distribution with a normalizing flow. Likewise any distribution can be mapped from a uniform distribution. Since both are true, then it is clear that a flow can map any distribution to any distribution. These proofs are similar in spirit to Sklar's theorem for copulas, as copulas share similar proprieties as normalizing flows. For example, copulas are monotonic transforms that map uniform distributions to arbitrary distributions.

lemma 4

The neural network parametrized by

$$G(X)_t = \sigma^{-1}(\sum_k w_k(x_1, ...x_{i-1}) * \sigma(\alpha_k(x_1, ...x_{i-1}) * x_i + \beta_k(x_1, ...x_{i-1})))$$

can transform a base distribution of an m dimensional uniform random variables to any desired m dimensional random variable conditioned on inputs with an index lower than the current index with arbitrary precision.

Here X is the uniform random variable and G transforms X into Y which is the target random variable that can be arbitrary in our proof. This equation is the same equation as in lemma 3, with σ being a sigmoid distribution. The one exception is the addition of the inverse sigmoid.

Proof:

Given an arbitrary ordering of the CDF of Y, defined as $F_t(y_t|x_1...x_{t-1})$, theorem 1 in Hyvärinen and Pajunen (1999), F(Y) is distributed uniformly in m dimensions. Then the inverse CDF function G_t is a monotonic function, as is $\sigma \circ G_t$. By lemma 3 $\sigma \circ G_t(F_t(y_t|x_1...X_{t-1}))$, can be arbitrarily approximated arbitrarily well by a sequence of bijectors with increasing K and N. The σ is

necessary because the preimage of $y_t(x_1...x_{t-1})$ in lemma 3 is (r_0, r_1). Thus $G_n = \sigma^{-1} \circ y(x)$ converges pointwise to G, as σ^{-1} is continuous. Since G_n converges to G and $G(X) = Y$, by lemma b we have $Y_n \to Y$.

QED

lemma 5

Given an arbitrary base distribution, a bijector can map this distribution to an arbitrary uniform distribution.

Define the CDF for all y dimeinsions as: $y_1 = F_1(x_1|0)$ and $y_t = F_t(x_t|y_1...y_{t-1})$. To be clear $y_t = Pr(X_t < x_t|y_1...y_{t-1})$. Again due to the theorem in Hyvärinen and Pajunen (1999), the distribution of the CDF is n dimensionally uniformly distributed. Then since lemma 3 shows that there exists a monotone function that converges pointwise to F and $F(X) = Y$, we have $Y_n \to Y$.

Theorem 6

Bijectors in the form,

$$H(X)_t = \sigma^{-1}(\sum_k w_k(x_1, ...x_{i-1}) * \sigma(\alpha_k(x_1, ...x_{i-1}) * x_i + \beta_k(x_1, ...x_{i-1})))$$

can map arbitrary distributions to arbitrary distributions. Again from theorem 1 from Hyvärinen and Pajunen (1999) in lemma 5 $F(X)$ is uniformly distributed so $G(F(X)) = Y$, from lemma 4. Since both functions are monotonic, a composition of the two functions is also a monotonic function and can be mapped via bijector. According to lemma 4, there exists a sequence of functions that converge to $\sigma \circ G \circ F$. Then there exists a $K_n = \sigma^{-1} \circ y(x)$ that converges pointwise to $G \circ F$. Then by lemma b we have $Y_n \to Y$.

Department of Economics, University of Michigan, Ann Arbor